第十一章：泛型与反射

1) 泛型的作用：1.限定向数组列表中添加的变量类型 2.获取变量的值不用强制转换（PPT 第 6 面）

2) 添加与泛型冲突的元素时，在**编译阶段**检测到非法的数据类型。（PPT 第 7 面）

3) 泛型类（至少要使用一个泛型的类，需要在开头就声明泛型）（PPT 第 8-12 面）

- 单个泛型类使用

```
class Generic<T> {
    public T t;
    public void fun(T t) {}
}

Generic<String> g1 = new Generic<>();
g1.t = "str";           //此处为合法数据类型
g1.t = 100;             //此处为非法数据类型

Generic<Integer> g2 = new Generic<>();
g2.t = 100;             //此处为合法数据类型
```

- 多个泛型类使用

```
class Generic<T, E> {
    public T t;
    public E e;
    public void fun(T t, E e) {}
}

Generic<String, Integer> g3 = new Generic<>();
g3.t = "str";
g3.e = 100;
```

4) 声明泛型不能用 int，需为 Integer（PPT 第 12 面）

5) 泛型方法（PPT 第 13-20 面）
泛型方法和泛型类没有关系，只有声明泛型的方法才是泛型方法

注意以下不是泛型方法

```
class Generic<T> {
    public T t;
    public void func(T t) {}
}
```

注意以下是泛型方法

```
class Generic<T> {
    public T t;
    public <T> void func(T t) {}
}
```

1. 调用泛型方法时，在方法名前的尖括号中填入具体类型。多数情况下，方法调用可以省略类型参数。

非泛型类中定义泛型方法

```
class GenericFun{
    public <T,E> void fun1(E e){}
    public <T> T fun2(T t){
        return t;
    }
}
```

- 调用泛型方法时，在方法名前的尖括号中填入具体类型。

```
GenericFun g = new GenericFun();
g.<String>fun2("str") );
```

- 多数情况下，方法调用可以省略类型参数。

```
g.fun2("str") );//返回值为str
```

2. 使用泛型方法，此处传入参数必须与泛型类声明的类型一致。（泛型类和泛型方法不冲突，与泛型类保持一致。）

泛型类中定义泛型方法

```
class GenericFun<K>{
    public <T> T fun2(T t,K k){
        return null;
    }
}
```

- 使用泛型方法，此处传入参数必须与泛型类声明的类型一致

```
GenericFun<Integer> g = new GenericFun<>();
g.fun2("str1",123) ;
```

3. 如果泛型方法的泛型与泛型类声明的泛型名称一致，则泛型方法中的泛型会覆盖类的泛型。（泛型类和泛型方法冲突，与泛型方法一致。）

```
class GenericFun<K>{
    public <T,K> T fun2(T t, K k){
        return null;
    }
}
```

- 使用泛型方法，此时类的泛型为Integer，泛型方法传入的参数为两个String

```
GenericFun<Integer> g = new GenericFun<>();
g.fun2("str1","str2") ;
```

4. 类的静态泛型方法，不得使用泛型类中声明的泛型，需要独立声明。

```
class GenericStaticMethod<K>{
    private K k;
    public GenericStaticMethod(K k){
        this.k=k;
}
    public static void fun1(K k){
        System.out.println(k)}
}//报错：无法从静态上下文中引用非静态类型变量K
    public static <K> void fun1(K k){
        System.out.println(k)}
}//可以编译成功
```



6) 泛型接口

定义以下泛型接口

```
interface GenericInterface <T>{
    T fun1();
}
```

实现类非泛型类，需具体指定接口的泛型

```
class GenericInterfaceImpl implements GenericInterface<String> {
    @Override
    public String fun1() {
        return null;
    }
}
```

实现类为泛型类，实现类的泛型要与接口一致

```
class GenericInterfaceImpl<T> implements GenericInterface<T> {
    @Override
    public T fun1() {
        return null;
    }
}
```

实现：

```
interface ShowInterface<T> { public void show(T t); }

//实现类已确定类型
class ShowClass1 implements ShowInterface<String>{
      public void show(String t){
            System.out.println(“show:”+t);
      }
}

//实现类未确定类型
Class ShowClass2<T> implements ShowInterface<T>{
      public void show(T t){
            System.out.println("show:"+t);
      }
}
```

使用：

```
public static void main(String[] args) {
      //实现类已确定类型
      ShowClass1 obj = new ShowClass1();
      obj.show("java");

      //实现类未确定类型，使用时确定
      ShowClass2<Integer> obj = new ShowClass2<>();
      obj.show(7);
}
```

7） 通配符（参数需要使用不确定类型，可以限定范围）

```
public static void printAllObject(List<?> list) {
    for (Object object : list) {
        System.out.println(object);
    }
}
public static void main(String[] args) {
    List<String> list1 = new Arraylist<>();
    List1.add(“java”);
    pritAllObject(list1);
    List<Interger> list2 = new Arraylist<>();
    List3.add(7);
    pritAllObject(list2);
}
```

❑ 上例（无限定通配符）中，如果想改成类型参数T的形式应该怎么改？

```
public static <T> void printAllObject(List<T> list)
```

允许类型参数发生变化
- <? extends ClassName>：类型参数是 ClassName 的子类
- <? super ClassName>：类型参数是 ClassName 的超类(父类)
- <?>：无限定通配符

8) 类型参数 T 与通配符？的区别

T 表示一个确定的类型，常用于泛型类和泛型方法的定义

?表示不确定的类型，不是类型变量，通常用于泛型方法的调用代码和形参，不能用于定义类和泛型方法。

9) 模板方法（行为型的类模式，反向控制结构）

抽象类（1 个）
- 定义并实现了一个模板方法，给出顶层逻辑的骨架
- 给出多个抽象方法
- 抽象方法中有些是钩子操作，即这些操作可以不被子类实现

具体类（n 个）
- 实现父类定义的一个或多个抽象方法

□模版方法模式的优缺点是什么？

优点：减少耦合（模板方法本身和这两个方法的具体实现之间被解耦了）；封装变化（子类封装了变化）。

缺点：带来额外的时间和空间开销。
- 每一个具体类都可以给出抽象方法的不同实现

10) 一个类只产生一个 Class 对象，类的类型对象存放在堆当中。

11) 获取 Class 类对象
方法一：对象.getClass()
方法二： Class.forName(类名的字符串)
方法三： T.class ，T 是任意的 Java 类型

12) 构造类的实例（记得强制转型）

方法一：使用 Class.newInstance（）

（适用于：类要有无参构造函数、类构造器不是 private）

```
Date date1 = new Date();
Class<?> dateClass2 = date1.getClass();
Date date2 = (Date)dateClass2.newInstance();
```

方法二：使用 Constructor 的 newInstance（任何情况都适用）

```
Student Harry = new Student("Harry Potter", 11);
Class<?> StudentClass = Harry.getClass();
Constructor<?> con = StudentClass.getDeclaredConstructor(String.class,
int.class);
Student Ron = (Student)con.newInstance("Ron Weasley", 11);
```

13) 获取和修改成员变量

```
Student Harry = new Student("Harry Potter", 11);
Class<?> StudentClass = Harry.getClass();
Field f = StudentClass.getDeclaredField("name");
f.setAccessible(true);//去除私有权限
Object v1 = f.get(Harry);//获取成员变量
f.set(Harry, “The boy who lived”);//修改成员变量
```

14) 获取成员方法（记得强制转型）

```
Student Harry = new Student("Harry Potter", 11);
Method m = Student.class.getMethod("getName");
String n = (String) m.invoke(Harry);
```

15) 反射优缺点

□优点

- 比较灵活，能够在运行时动态获取类的实例

□缺点

- 性能瓶颈：反射相当于一系列解释操作，通知JVM要做的事情，性能比直接的Java代码要慢很多
- 安全问题：反射机制破坏了封装性，因为通过反射可以获取并调用类的私有方法和字段。

16）设计安全的全局单例（饿汉式）

单例

- 单例的目的是保证某个类仅有一个实例
  - 在类被加载时就实例化一个对象

```
public class Singleton {
    private static Singleton singleton = new Singleton();
    private Singleton(){}

    public static Singleton getInstance(){
        return singleton;
    }
}
```

68

利用反射破坏单例

- 通过反射中的setAccessible(true)覆盖Java中的访问控制，进而调用私有的构造函数

```
public class SingletonTest {
    public static void main(String[] args) {
        Class<?> objectClass = Singleton.class;
        Constructor<?> constructor = objectClass.getDeclaredConstructor();
        constructor.setAccessible(true);
        Singleton instance = Singleton.getInstance();
        Singleton newInstance = (Singleton)constructor.newInstance();
        System.out.println(instance == newInstance);
    }
}// instance和newInstance是不同的实例 False
```

69

抵御反射破坏

```
public class Singleton {
    private static Singleton singleton = new Singleton();
    private Singleton(){
        if(singleton != null) {
            throw new RuntimeException("单例构造器禁止通过反射调用");
        }
    }
    public static Singleton getInstance(){
        return singleton;
    }
}
```

70